

IGNITER FRONTEND

Comprehensive Technical Guide & Architecture Manual

An in-depth guide covering every frontend feature, React architectural pattern, React hooks deep dive (Where, Why, and How), centralized Axios networking layer, and UI state synchronization. Exclusively focused on the frontend codebase.

FRAMEWORK	ROUTING	HTTP CLIENT	LANGUAGE & BUILD
React 19.2.8	React Router v7	Axios 1.20	TypeScript + Vite

TABLE OF CONTENTS

1. Application Overview & Architecture

2. Main Frontend Features

3. Complete React Hooks Deep Dive
4. React Router v7 Navigation

5. Axios Networking & API Layer

6. Native Web APIs & State Patterns

1. Application Architecture & Overview

Igniter is a reactive, social microblogging web application modeled after modern feed platforms (such as Meta Threads and X/Twitter). The frontend provides a fast single-page application (SPA) experience with client-side routing, optimistic UI updates, cursor-based and offset pagination, real-time debounced search, nested commenting hierarchies, and background notification polling.

Frontend File & Directory Structure

Directory / File	Type	Technical Responsibility
<code>src/main.tsx</code>	Application Entrypoint	Bootstraps React 19 root using <code>createRoot()</code> and wraps the app in <code><StrictMode></code> .
<code>src/App.tsx</code>	Root Controller Component	Manages JWT authentication state, listens for global storage/custom events, and guards route rendering.
<code>src/api/Axios.ts</code>	Axios Client Core	Central Axios instance configured with conditional <code>baseUrl</code> and request interceptor for JWT injection.
<code>src/api/*Api.ts</code>	API Service Layer	Modular endpoint abstractions: <code>AuthApi</code> , <code>UserApi</code> , <code>ThreadApi</code> , <code>CommentApi</code> , and <code>NotificationApi</code> .
<code>src/components/Sidebar.tsx</code>	Navigation Component	Provides desktop header, mobile bottom navigation dock, unread notification counter badge, and background polling.
<code>src/components/CommentSection.tsx</code>	Hierarchical Comments	Two-tier threaded commenting engine supporting root comments, nested replies, inline composer, and autofocus.
<code>src/pages/Threads.tsx</code>	Feed Page	Cursor-based feed with infinite scrolling (<code>IntersectionObserver</code>), thread creation, editing, deletion, and liking.
<code>src/pages/Profile.tsx</code>	User Profile Page	Own vs. target user profile inspection, bio/username editing, avatar upload, follow/unfollow modal, and tabbed thread lists.
<code>src/pages/Search.tsx</code>	Discovery Page	Debounce 220ms user search with real-time follow status querying, plus keyword-filtered client thread exploration.
<code>src/pages/Notifications.tsx</code>	Activity Page	Categorized notification cards (LIKE, FOLLOW, COMMENT) with optimistic read toggling and "Mark all read" action.
<code>src/pages/Login.tsx</code>	Authentication Portal	Tabbed interface for Login vs. Registration with validation constraints and auto-login upon account creation.
<code>src/types/*.ts</code>	TypeScript Contracts	Strict type definitions for <code>User</code> , <code>Thread</code> , <code>Comment</code> , <code>Notification</code> , and pagination interfaces.

2. Main Frontend Features & Functional Breakdown

The application implements comprehensive client-side functionality engineered with clean state transitions, optimistic rendering, and clear feedback loops:

2.1 Authentication & Multi-Tab Synchronization

- **Dual-Mode Auth Portal (`Login.tsx`):** Allows users to toggle seamlessly between *Sign In* and *Create Account*. Includes client-side pre-validation matching backend schema rules (e.g., username length 3–10 characters, password minimum 4 characters).
- **Automated Post-Registration Session:** When a user signs up via `createUser()`, the component automatically executes `login()` with the new credentials and stores the resulting JWT token in `localStorage`.
- **Multi-Tab Session Sync (`App.tsx`):** Listens to both browser `storage` events and custom `"auth-token-changed"` events. If a user logs in or logs out in one browser tab, all other open tabs immediately synchronize their authentication state without a full page reload.
- **Protected Route Gateway: `App.tsx`** conditionally renders the protected application shell (Sidebar + authenticated routes) or redirects unauthenticated visitors to `/login` using React Router's `<Navigate to="/login" />`.

2.2 Main Feed & Infinite Scrolling (`Threads.tsx`)

- **Cursor-Based Pagination:** Fetches threads using `getFeedCursorPage(cursor, 10)`. Unlike rigid offset pagination, cursor pagination prevents duplicate or skipped items as new threads are posted dynamically.
- **Infinite Scroll via `IntersectionObserver`:** A sentinel DOM element (`<div ref={sentinelRef} />`) is monitored. When scrolled into viewport range (`rootMargin: "200px"`), `loadNextPage()` is triggered automatically.
- **Manual "Load More" Fallback:** Provides an accessible button with a loading spinner if the user prefers manual paging or if intersection hasn't fired yet.
- **Optimistic Thread Publishing:** Upon submitting a new thread, the created item is immediately prepended to the top of the local `threads` state array before awaiting subsequent feed refreshes.
- **In-Place Thread Editing & Deletion:** Thread authors can toggle an in-place editing textarea, modify thread content, and save or cancel. Deletion uses `window.confirm` followed by immediate state filtering.

2.3 Like & Engagement Engine

- **Optimistic Heart Toggle:** Clicking the heart icon immediately toggles the active visual state, adjusts the like count, and sends an asynchronous request to `likeThread()` or `unlikeThread()`.
- **Per-User LocalStorage Persistence:** Likes are cached under `liked_threads_${username}` so like status remains persistent across page transitions, user profiles, and searches.
- **Stale Like Cleanup: `Profile.tsx`** automatically reconciles stored like IDs against system threads, pruning any orphaned or desynchronized entries where like count is 0.

2.4 Nested & Threaded Comments (`CommentSection.tsx`)

- **Two-Level Tree Structure:** Distinguishes between root thread comments and nested replies via `parentCommentId`. Replies are grouped by parent ID and rendered within a sub-tree with a visual connector line.
- **Inline Reply Composer:** Clicking "Reply" on any comment reveals an inline composer prefilled with `@username`, renders a "Replying to @username" pill badge, and automatically focuses the input using `useRef`.

- **Dynamic Comment Count Uplift:** Whenever comments or replies are posted or deleted, `onCommentCountChange(count)` propagates the new count back to the parent card in `Threads.tsx`, `Profile.tsx`, or `Search.tsx`.
- **Notification Deep Linking:** Supports `highlightCommentUsername` prop to highlight specific commenter replies with a "From notification" badge.

2.5 User Profiles & Social Graph (`Profile.tsx`)

- **Contextual Rendering (Self vs. Other):** Reads URL query parameters via `useSearchParams()`. If `?username=foo` is present and differs from current user, it renders the public profile of user `foo` with a Follow/Unfollow button and disables profile editing.
- **Profile Image Upload via Multipart FormData:** Clicking the camera badge opens the native OS file picker. Selected files are wrapped into a `FormData` instance and dispatched via `uploadProfilePicture(file)`.
- **Interactive Followers/Following Modal:** Clicking follower or following counters opens a modal displaying users with instant follow/unfollow toggle actions for each listed person.
- **Profile Tabbed Navigation:** Toggles between user-created *Threads* (with offset pagination) and *Liked* posts.

2.6 Debounced Discovery & Search (`Search.tsx`)

- **Real-Time Character-by-Character Querying:** Dispatches search requests as users type, throttled by a 220ms debounce timer via `useRef` to prevent API flood.
- **Concurrent UI Rendering with `useTransition`:** Wraps user search state updates in `startTransition` to keep the text input responsive while rendering user card lists.
- **Live Follow Status Mapping:** When user results return, parallel promises fetch the current user's follow status for each discovered account, displaying "Follow" or "Following" accurately.
- **Client-Side Posts Filter:** Pre-loads threads and executes instantaneous keyword filtering on thread text and author names.

2.7 Notification Center (`Notifications.tsx` & `Sidebar.tsx`)

- **Visual Categorization:** Badges each notification by type (red Heart for `LIKE`, purple UserPlus for `FOLLOW`, blue MessageCircle for `COMMENT`).
- **Optimistic Read Status:** Clicking an unread item immediately clears the unread dot and triggers `markNotificationAsRead()`.
- **Batch Read:** "Mark all read" updates all items in state and dispatches parallel PUT requests.
- **Cross-Component Sync:** Dispatches `"notifications-updated"` custom DOM event so the navigation badge in `Sidebar.tsx` updates immediately.
- **Background Polling:** `Sidebar.tsx` executes background polling every 15 seconds to keep unread badges fresh.

3. Complete React Hooks Deep Dive (Where, Why & How)

Every standard and React Router hook utilized in the Igniter frontend is broken down below with exact file locations, architectural rationale, and verified code implementations.

1. `useState<T>(initialState)`

CORE REACT HOOK

WHERE IS IT USED?

Used across almost every component and page in the project: `App.tsx` (lines 15–17), `Sidebar.tsx` (line 16), `CommentSection.tsx` (lines 54–63), `Threads.tsx` (lines 44–63), `Profile.tsx` (lines 72–113), `Search.tsx` (lines 29–49), `Notifications.tsx` (lines 40–41), and `Login.tsx` (lines 15–28).

WHY IS IT USED?

To declare reactive state variables that trigger component re-renders when modified. It holds local form inputs, async data arrays (threads, comments, users), loading flags, error strings, active tab selections, pagination cursors, and modal visibility states.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// 1. Lazy state initialization in App.tsx (reads localStorage once on mount):
const [token, setToken] = useState<string | null>(() =>
  localStorage.getItem("token")
);

// 2. Set-based tracking in Threads.tsx (manages which thread cards have comments open):
const [openCommentThreadIds, setOpenCommentThreadIds] = useState<Set<number>>(new Set());

// 3. Functional updater to prevent race conditions during toggles:
const toggleComments = (threadId: number) => {
  setOpenCommentThreadIds((prev) => {
    const next = new Set(prev);
    if (next.has(threadId)) next.delete(threadId);
    else next.add(threadId);
    return next;
  });
};
```

2. useEffect(setup, dependencies?)

CORE REACT HOOK

WHERE IS IT USED?

`App.tsx` (lines 19–31), `Sidebar.tsx` (lines 18–46), `CommentSection.tsx` (lines 66–97), `Threads.tsx` (lines 157–188), `Profile.tsx` (lines 219–241, 334–336), `Search.tsx` (lines 54–168), `Notifications.tsx` (lines 55–57).

WHY IS IT USED?

To synchronize components with external systems: fetching API data on mount or parameter changes, registering global DOM event listeners, polling timers, managing `IntersectionObserver` for infinite scrolling, and managing element focus.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// App.tsx: Listening for multi-tab auth changes and cleaning up listeners on unmount
useEffect(() => {
  const handleAuthTokenChanged = () => {
    setToken(localStorage.getItem("token"));
  };

  window.addEventListener("auth-token-changed", handleAuthTokenChanged);
  window.addEventListener("storage", handleAuthTokenChanged);

  return () => {
    window.removeEventListener("auth-token-changed", handleAuthTokenChanged);
    window.removeEventListener("storage", handleAuthTokenChanged);
  };
}, []);

// Sidebar.tsx: Polling unread notifications every 15s with isMounted cancellation guard:
useEffect(() => {
  let isMounted = true;
  const fetchUnread = async () => {
    try {
      const count = await getUnreadNotificationCount();
      if (isMounted) setUnreadCount(count);
    } catch {}
  };

  fetchUnread();
  const interval = setInterval(fetchUnread, 15000);

  return () => {
    isMounted = false;
    clearInterval(interval);
  };
}, [location.pathname]);
```

3. useRef<T>(initialValue)

CORE REACT HOOK

WHERE IS IT USED?

- `CommentSection.tsx` (line 64): `replyInputRef = useRef<HTMLInputElement>(null)`
- `Threads.tsx` (line 66): `sentinelRef = useRef<HTMLDivElement | null>(null)`
- `Profile.tsx` (line 96–97): `sentinelRef` and `threadsSectionRef`
- `Search.tsx` (line 51): `debounceTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null)`

WHY IS IT USED?

`useRef` serves two distinct critical roles in Igniter:

1. **Direct DOM Element Access:** To manipulate DOM nodes directly (e.g., calling `.focus()` on the reply input when replying starts, or passing the sentinel `<div>` to `IntersectionObserver.observe()`).
2. **Mutable Value Persistence:** To persist mutable timer references (like `debounceTimerRef`) across re-renders without causing unwanted re-renders when updated.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// 1. Autofocus reply input when user clicks 'Reply' on a comment (CommentSection.tsx):
const replyInputRef = useRef<HTMLInputElement>(null);

useEffect(() => {
  if (replyingTo && replyInputRef.current) {
    replyInputRef.current.focus();
  }
}, [replyingTo]);

// 2. Debounce timer for search without triggering extra renders (Search.tsx):
const debounceTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);

if (debounceTimerRef.current) clearTimeout(debounceTimerRef.current);
debounceTimerRef.current = setTimeout(async () => {
  const results = await searchUsers(trimmed);
  setUsers(results);
}, 220);
```

4. useCallback(fn, dependencies)

CORE REACT HOOK

WHERE IS IT USED?

- `Threads.tsx` (lines 126–155): `loadNextPage` pagination callback.
- `Profile.tsx` (lines 174–216): `loadNextThreadsPage` offset pagination callback.

WHY IS IT USED?

To memoize infinite scrolling handler functions. Because these pagination functions are passed as dependencies to the `useEffect` that sets up the `IntersectionObserver`, wrapping them in `useCallback` prevents the observer from being detached, destroyed, and re-instantiated on every minor component re-render.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// Threads.tsx: Memoized pagination function connected to IntersectionObserver
const loadNextPage = useCallback(async () => {
  if (loadingMore || !hasNext || !cursor || loading) return;

  setLoadingMore(true);
  try {
    const pageData = await getFeedCursorPage(cursor, PAGE_SIZE);
    setThreads((prev) => {
      const existingIds = new Set(prev.map((t) => t.id));
      const filtered = pageData.content.filter((t) => !existingIds.has(t.id));
      return [...prev, ...filtered];
    });
    setCursor(pageData.nextCursor);
    setHasNext(pageData.hasNext);
  } finally {
    setLoadingMore(false);
  }
}, [loadingMore, hasNext, cursor, loading, currentUsername]);
```

5. useTransition()

REACT 19 CONCURRENT HOOK

WHERE IS IT USED?

- `Search.tsx` (line 31 & lines 104–106).

WHY IS IT USED?

To mark the rendering of search results as a low-priority, non-blocking transition. When users type rapidly into the search box, updating the input value is urgent, whereas rendering the matching user cards is secondary.

- `startTransition` ensures the browser input never drops frames or feels sluggish.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// Search.tsx:
const [, startTransition] = useTransition();

// Inside debounced search response handler:
const results = await searchUsers(trimmed);
startTransition(() => {
  setUsers(results); // Non-blocking UI update
});
```


6. `useNavigate()`, `useLocation()`, `useSearchParams()`

REACT ROUTER V7 HOOKS

WHERE AND WHY ARE THEY USED?

- `useNavigate()` in `Login.tsx` & `Profile.tsx`: Used for programmatic navigation after state changes (e.g., navigating to `/` immediately after successful login).
- `useLocation()` in `Sidebar.tsx`: Inspects `location.pathname` to determine which navigation icon should be highlighted with the active styling class (`active`) and refresh unread badges when switching routes.
- `useSearchParams()` in `Profile.tsx`: Reads URL query parameters (e.g. `/profile?username=john_doe`). If `searchParams.get("username")` exists, the component displays the requested user's public profile instead of the logged-in user.

HOW IS IT USED IN IGNITER? (CODE SNIPPET & MECHANICS)

```
// 1. useNavigate in Login.tsx:
const navigate = useNavigate();
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  const data = await login({ username, password });
  localStorage.setItem("token", data.token);
  navigate("/"); // Imperative navigation
};

// 2. useLocation in Sidebar.tsx:
const location = useLocation();
const isFeedActive = location.pathname === "/";

// 3. useSearchParams in Profile.tsx:
const [searchParams] = useSearchParams();
const viewUsername = searchParams.get("username");
useEffect(() => {
  if (viewUsername) {
    // Load target user's public profile
  } else {
    // Load current authenticated user's profile
  }
}, [viewUsername]);
```

4. React Router v7 Declarative Routing Architecture

The application implements an authentication-driven route branching architecture in `src/App.tsx`:

```
<BrowserRouter>
  {token ? (
    <div className="app">
      <Sidebar />
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Threads />} />
          <Route path="/threads" element={<Navigate to="/" />} />
          <Route path="/search" element={<Search />} />
          <Route path="/notifications" element={<Notifications />} />
          <Route path="/profile" element={<Profile />} />
          <Route path="/login" element={<Navigate to="/" />} />
          <Route path="*" element={<Navigate to="/" />} />
        </Routes>
      </main>
    </div>
  ) : (
    <Routes>
      <Route path="/login" element={<Login onLoginSuccess={handleLoginSuccess} />} />
      <Route path="*" element={<Navigate to="/login" />} />
    </Routes>
  )}
</BrowserRouter>
```

Key Routing Design Principles:

- Authentication Gateway:** Instead of wrapping individual routes in complex higher-order components, `App.tsx` uses a root ternary check on `token`. If authenticated, the layout shell with `Sidebar` and authenticated routes is mounted. If unauthenticated, all wildcard paths redirect to `/login`.
- Alias & Redirect Handling:** Navigating to `/threads` redirects directly to `/`, avoiding duplicate route configurations. Authenticated visits to `/login` automatically bounce to the feed (`/`).
- Query Parameter Deep-Linking:** `/profile` serves both the personal profile and third-party profiles via query strings (`/profile?username=testuser`), avoiding route duplication.

5. Axios Networking Architecture & API Layer

All network traffic in Igniter is routed through a single, customized Axios instance defined in `src/api/Axios.ts`. This ensures consistent base URLs, security headers, token attachment, and proxy handling.

5.1 The Axios Instance & Request Interceptor

```
import axios from "axios";

// Determine if executing on localhost or over a tunnel / remote proxy
const isLocalhost =
  typeof window !== "undefined" &&
  (window.location.hostname === "localhost" ||
    window.location.hostname === "127.0.0.1");

const api = axios.create({
  baseURL: isLocalhost ? "http://localhost:8080" : "",
});

// Request Interceptor: Automatic JWT token & proxy header injection
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  // Bypass ngrok interstitial warning page for smooth mobile/remote dev testing
  config.headers["ngrok-skip-browser-warning"] = "true";

  return config;
});

export default api;
```

5.2 Detailed API Modules & Methods

Module	Method & Route	Payload / Parameters	Functional Behavior
AuthApi	POST /api/auth/login	{ username, password }	Authenticates credentials and returns JWT bearer token.
AuthApi	POST /api/auth/logout	None	Calls backend logout, clears <code>localStorage</code> , and redirects to <code>/login</code> .
UserApi	POST /api/users	{ username, email, password, bio }	Registers a new user account with validation.
UserApi	GET /api/users/me	None (Bearer token)	Returns profile of current user; runs through <code>normalizeUser()</code> .
UserApi	PUT /api/users/me	{ username, email, bio }	Updates current user bio and credentials.
UserApi	POST /api/users/profile-picture	FormData: file	Uploads binary avatar image with <code>multipart/form-data</code> .
UserApi	GET /api/users/search	params: { text }	Searches users matching query characters in real-time.
UserApi	POST /api/users/:id/follow	None	Follows target user; updates social graph.
UserApi	POST /api/users/:id/unfollow	None	Unfollows target user.
ThreadApi	GET /api/threads/feed	params: { cursor, size }	Returns cursor-paginated feed with <code>nextCursor</code> and <code>hasNext</code> .
ThreadApi	POST /api/threads/create	{ content }	Creates a new thread post.
ThreadApi	PUT /api/threads/:id	{ content }	Updates existing thread content.
ThreadApi	DELETE /api/threads/:id	None	Deletes thread post.
ThreadApi	POST /api/threads/:id/like	None	Likes thread; returns updated <code>liked</code> state and <code>likeCount</code> .
ThreadApi	DELETE /api/threads/:id/like	None	Unlikes thread; returns updated like count.
CommentApi	GET /api/threads/:id/comments	None	Retrieves all comments and nested replies for given thread.

Module	Method & Route	Payload / Parameters	Functional Behavior
CommentApi	POST /api/threads/:id/comments	{ content, parentCommentId }	Creates root comment or nested reply depending on parentCommentId .
CommentApi	DELETE /api/threads/comments/:id	None	Deletes comment (backend cascades to its child replies).
NotificationApi	GET /api/notifications	None	Retrieves list of user notifications.
NotificationApi	GET /api/notifications/unread/count	None	Fetches numerical unread counter for navigation badge.
NotificationApi	PUT /api/notifications/:id/read	None	Marks single notification as read.

5.3 Special Axios Techniques Used in the Project

1. Multipart File Upload via FormData:

In `UserApi.ts`, avatar uploads wrap standard browser `File` objects into a native `FormData` object before dispatching via Axios:

```
export const uploadProfilePicture = async (file: File): Promise<User> => {
  const formData = new FormData();
  formData.append("file", file);

  // Axios automatically sets Content-Type: multipart/form-data with boundary
  const response = await api.post<User>("/api/users/profile-picture", formData);
  return normalizeUser(response.data);
};
```

2. Defensive URL Normalization (`normalizeUser`):

Because backend servers might return absolute `http://localhost:8080/uploads/...` paths or relative paths, `UserApi.ts` sanitizes profile picture paths so Vite proxy forwarding handles them without CORS or mixed-content SSL errors:

```
export const normalizeUser = (user: User): User => {
  if (!user) return user;
  let pic = user.profilePicture;
  if (pic && typeof pic === "string") {
    if (pic.startsWith("http://localhost:8080/")) {
      pic = pic.replace("http://localhost:8080/", "/");
    } else if (!pic.startsWith("/") && !pic.startsWith("http")) {
      pic = `/${pic}`;
    }
  }
  return { ...user, profilePicture: pic };
};
```

3. Spring Data Page Response Normalizer:

In `ThreadApi.ts`, `parsePageResponse()` safely parses various pagination payloads (arrays, Spring Page objects, or empty results), preventing runtime undefined access errors across pagination renders.

6. Native Web APIs, State Patterns & Styling Architecture

6.1 Browser Web APIs Integrated with React

- IntersectionObserver API:** Orchestrates high-performance infinite scrolling in `Threads.tsx` and `Profile.tsx` by notifying React when an off-screen sentinel element enters the viewport.
- CustomEvent & Event Dispatching:** Decouples unrelated components using custom events such as `window.dispatchEvent(new Event("auth-token-changed"))` and `new CustomEvent("notifications-updated")`.
- localStorage:** Persists auth tokens and per-user liked thread IDs across page refreshes.
- setInterval & clearInterval:** Powers real-time background notification polling in `Sidebar.tsx`, cleaned up on unmount to avoid memory leaks.

6.2 Styling & Iconography Architecture

- **Vanilla CSS Design System** (`App.css` & `index.css`): Crafted with CSS custom properties (variables), high-contrast dark theme aesthetics, glassmorphism cards, and fluid layouts.
- **Lucide React Icons**: Utilizes tree-shakable SVG icons (`Flame` , `Heart` , `MessageCircle` , `Search` , `User` , `LogOut` , `Loader2`) with programmatic sizing, fill colors, and stroke widths.
- **Micro-Animations**: CSS keyframe animations (e.g. `.spin-icon { animation: spin 1s linear infinite; }`) provide smooth loading indicators during async operations.

Summary Checklist of Frontend Engineering Patterns

- ✓ **React 19 Hooks**: `useState` , `useEffect` , `useRef` , `useCallback` , `useTransition` .
- ✓ **React Router v7**: `useNavigate` , `useLocation` , `useSearchParams` , `BrowserRouter` , `Routes` , `Route` , `Navigate` .
- ✓ **Axios HTTP Client**: Central instance, dynamic baseURL, request interceptor with JWT injection, multipart FormData uploads, Spring Data parsing.
- ✓ **Event-Driven State**: Cross-tab storage listeners, custom DOM events for auth and badge updates.
- ✓ **User Experience**: Cursor-based infinite scrolling, debounced live search, optimistic like/read toggles, and nested comments.